

Software-Architektur als Schichtendarstellung

Analyse von Software-Architektur mit S202

Johannes Weigend

Finale Fassung, 30. Mai 2026

Zusammenfassung

S202 berechnet aus Bytecode eine hierarchische Schichtendarstellung von Java-Systemen. Das Werkzeug leitet eine Architekturhypothese aus den beobachteten Klassenabhängigkeiten ab, macht Zyklen und Rückkanten als explizite Befunde sichtbar und prüft die fertige Darstellung redundant gegen Konsistenzregeln. Dieses Dokument beschreibt die typischen Fehler bei der Berechnung solcher Layouts, erklärt die konzeptionelle Lösung und zeigt, wie die Darstellung als Arbeitswerkzeug für Refactoring-Planung genutzt werden kann.

1 Einleitung

Spätestens seit den Schichtenmodellen für Betriebssysteme Ende der 1960er-Jahre wird Software-Architektur als geschichtete Abhängigkeitsstruktur dargestellt. Dieses Prinzip ist bis heute aktuell: Es hilft in Cloud-Native-Architekturen, technische Abhängigkeiten zwischen Diensten, Plattformen und Infrastruktur zu ordnen, und im Domain-driven Design, fachliche Grenzen und Abhängigkeiten zwischen Bounded Contexts sichtbar zu machen. Die Grundidee ist einfach: Ein Element, das ein anderes Element benutzt, steht oberhalb des benutzten Elements. Abhängigkeiten laufen damit von oben nach unten. Kanten in Gegenrichtung sind auffällig, weil sie häufig auf Zyklen, unerwünschte Kopplung oder eine gebrochene Schichtenordnung hinweisen.

Solche vertikal ausgerichteten Graphen helfen, Systeme zu verstehen und Anomalien in ihren Abhängigkeiten zu erkennen. Ein einfaches Beispiel liefern das Java-Werkzeug `jdeps` und das Graphviz-Werkzeug `dot`: `jdeps` extrahiert die Klassenabhängigkeiten aus einem Jar, `dot` erzeugt daraus einen gerichteten Graphen.

Abbildung 1 zeigt ein azyklisches Beispiel aus dem Example-Jar der S202-Codebasis. Erstellt mit `jdeps` und `dot`. Das Jar enthält neun Klassen in drei Paketen: `example`, `example1` und `example2`.

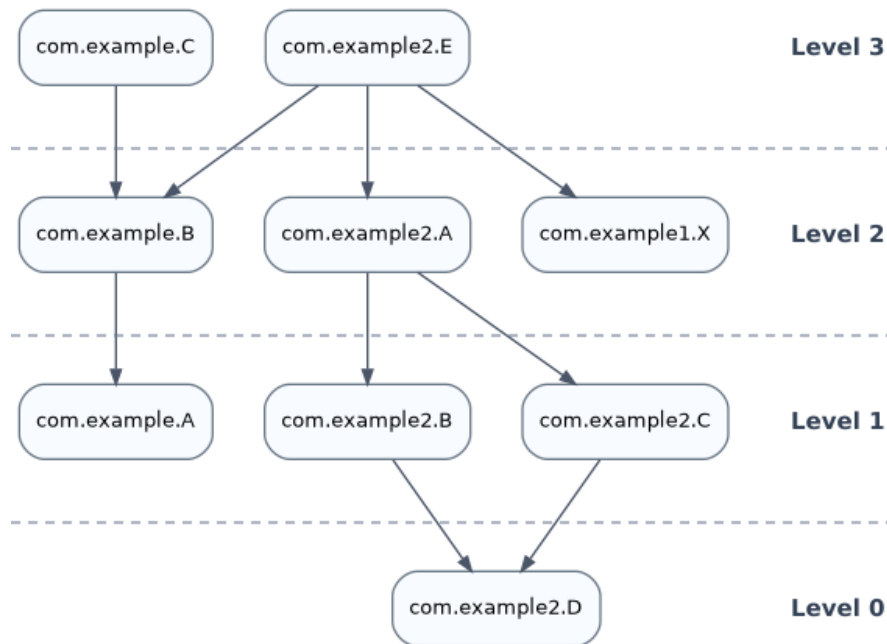


Abbildung 1: Azyklisches Beispiel aus `test-example-1.0.0.jar`.

Die Pfeile zwischen den Boxen sind Abhängigkeiten von Klassen durch Aufruf oder Nutzung. Im Beispiel nutzt `com.example.B` die Klasse `com.example.A`, kurz $B \rightarrow A$. Die horizontalen Linien sind manuell ergänzt und visualisieren die Ebenen der Schichtung. Level 0 bildet das Fundament; höhere Levels stehen für Elemente, die von darunterliegenden Elementen abhängen.

Diese Art der Darstellung ist nützlich und weit verbreitet. Als Visualisierung realer Anwendungen hat sie aber drei praktische Probleme:

1. Viele Pfeile machen die Grafik schon bei mittelgroßen Anwendungen unübersichtlich.
2. Die Paketstruktur ist über mehrere Levels verteilt und bildet keine sichtbare Gruppierung.
3. Bei zyklischen Abhängigkeiten muss der Zyklus für die Levelberechnung aufgebrochen werden, damit überhaupt eine vertikale Ordnung entsteht. Der ursprüngliche Abhängigkeitsgraph bleibt unverändert; verändert wird nur der daraus abgeleitete DAG (*Directed Acyclic Graph*, ein gerichteter Graph ohne Zyklen), auf dem die Levels berechnet werden. Entscheidend ist deshalb, welche Kante für diese Berechnung ausgeklammert wird, denn genau diese Kante erscheint später als Bruch der berechneten Ordnung.

2 S202 als hierarchische Schichtendarstellung

Mit dem Open-Source-Werkzeug S202 versuchen wir, diese Probleme gezielt zu adressieren. S202 ersetzt den flachen Klassengraphen nicht einfach durch eine verbesserte Darstellung, sondern durch eine hierarchische Architekturhypothese: Pakete bleiben als Container sichtbar, und innerhalb jedes Containers werden Pakete oder Klassen nach ihren Abhängigkeiten angeordnet.

Die Pfeile können optional eingeblendet werden. Im Normalfall soll die vertikale Ordnung aber bereits ohne Pfeile verständlich machen, welche Teile der Anwendung weiter oben liegen und welche als Fundament dienen. Unerwünschte Abhängigkeiten fallen dadurch nicht als einzelne Linie in einem großen Netz auf, sondern als Bruch in einer erwarteten Schichtenordnung.

Im Gegensatz zum flachen Klassengraphen sind die Levels bei S202 hierarchisch durch die Paketstruktur gegliedert. Jedes Paket ordnet seine direkten Kinder, also Subpakete oder Klassen, nach deren lokalen Abhängigkeiten. Die Paketstruktur bleibt dadurch sichtbar, statt im Klassengraphen auseinandergezogen zu werden.

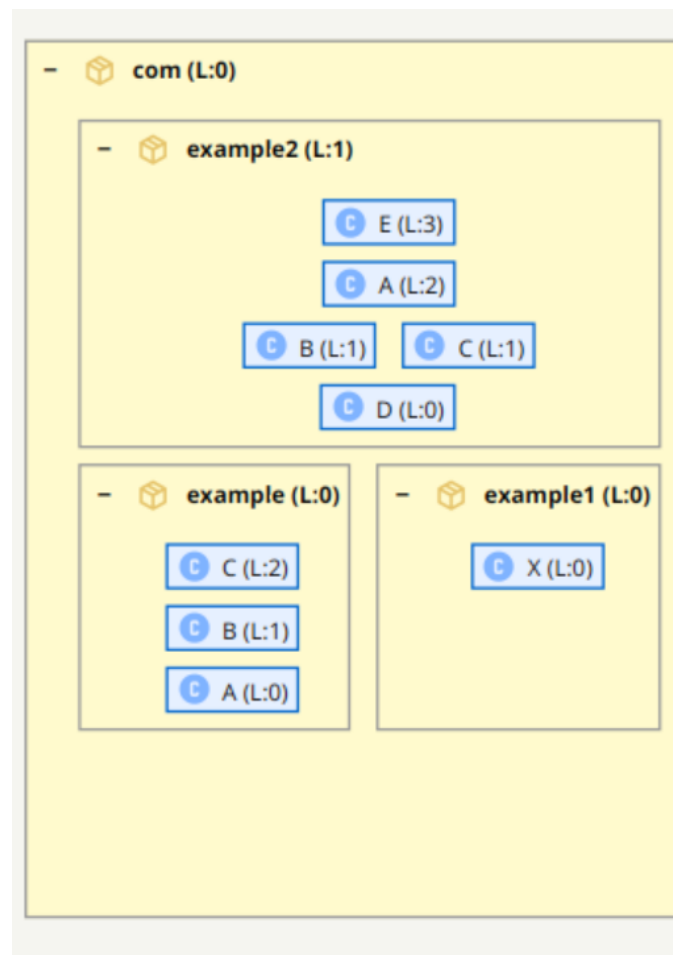


Abbildung 2: S202-Darstellung des azyklischen Beispielausschnitts: Pakete bleiben als Container sichtbar, Klassen werden innerhalb ihres Containers geordnet, und Pfeile können bei Bedarf eingeblendet werden.

Das Beispiel zeigt die Paketstruktur unterhalb des äußeren Pakets `com`: `example`, `example1` und `example2` sind Teilbäume dieses Containers. Im Teilbaum `example2` gibt es eine nicht triviale innere Klassenstruktur; zugleich zeigt die übergreifende Struktur, dass `example2` von `example` und `example1` abhängt. Auf Klassenebene gilt für `example2`: $E \rightarrow A \rightarrow (B \mid C) \rightarrow D$. Die Darstellung abstrahiert dabei bewusst von einzelnen Kanten. Ob `A` eine Abhängigkeit nach `B`, nach `C` oder nach beiden hat, ist für die grobe Architekturansicht weniger wichtig als die Schichtung des Pakets. Genau diese Abstraktion macht umfangreiche Beziehungsnetze noch lesbar.

Die Idee ist nicht neu. Das kommerzielle Produkt Structure101 hatte bereits eine sehr ähnliche Darstellung. Es gibt jedoch keine offene Implementierung und keine technische Dokumentation, die erklärt, wie man ein solches Layout zuverlässig berechnet. S202 schließt diese Lücke als

Open-Source-Werkzeug unter der Apache-Lizenz.

Dieses Dokument beschreibt die typischen Fallstricke bei der Realisierung eines solchen Layouts und zeigt eine Lösung, mit der sich viele dieser Probleme konzeptionell sauber eliminieren lassen.

3 Typische Fehler bei der Berechnung von Schichten

Die Abbildungen 1 und 2 zeigen bewusst ein einfaches, azyklisches Beispiel. Für diesen Fall ist die Schichtung eindeutig: E steht über A, A steht über B und C, und beide stehen über D. Es gibt keine Rückkopplung, deshalb muss keine Kante ausgewählt werden, die für die Levelberechnung ignoriert wird.

In realen Anwendungen ist dieser Fall eher die Ausnahme. Sobald Klassen oder Pakete zyklisch voneinander abhängen, reicht ein normaler Durchlauf über den Graphen nicht mehr aus. Genau dort beginnen die interessanten Fehler: Die Darstellung sieht oft noch plausibel aus, aber die berechneten Ebenen sind nicht mehr stabil oder nicht mehr fachlich erklärbar.

In den folgenden Abschnitten spielt der Begriff SCC eine zentrale Rolle. Eine SCC (*Strongly Connected Component*, stark zusammenhängende Komponente) ist eine Menge von Knoten, in der jeder Knoten jeden anderen über gerichtete Abhängigkeitspfade erreichen kann. In einem solchen Zyklus können nicht alle Abhängigkeiten gleichzeitig nach unten laufen.

Ein korrektes Schichtlayout muss deshalb gleichzeitig mehrere Anforderungen erfüllen:

1. Wenn $A \rightarrow B$ gilt, muss A oberhalb von B stehen.
2. Pakete müssen als Container sichtbar bleiben. Ein Paket darf nicht zufällig verschoben werden, nur weil eine einzelne enthaltene Klasse ein hohes Klassenlevel hat.
3. Klassen oder Pakete, die eine SCC bilden, müssen explizit behandelt werden, weil in einem Zyklus nicht alle Kanten gleichzeitig nach unten laufen können.

Diese Anforderungen sind einzeln gut lösbar. Zusammen erzeugen sie aber ein zirkuläres Abhängigkeitssystem zwischen den Ebenen der Berechnung selbst: Klassenlevel beeinflussen sichtbare Positionen, Paketcontainer strukturieren diese Positionen, und SCCs erzwingen Korrekturen an einer Ordnung, die man ohne Zyklusbehandlung schon berechnet hätte. Genau daraus entstehen die typischen Fehlermodi.

3.1 Retroaktive Korrektur

Retroaktive Korrektur bedeutet hier: Eine später erkannte Struktur erzwingt eine nachträgliche Änderung an bereits berechneten Levels.

Ein naiver Algorithmus läuft über die Klassen und vergibt direkt Level. Er sieht zuerst Klasse A, setzt ihr Level und hebt damit auch das Paket von A an. Später findet er Klasse B und erkennt: A und B gehören zu derselben SCC und müssen gemeinsam behandelt werden. Jetzt müsste A korrigiert werden, und damit auch ihr Paket und möglicherweise weitere Pakete, die davon abhängen.

Das Problem ist nicht die Korrektur selbst. Das Problem ist, dass Teile des Layouts zu diesem Zeitpunkt bereits als fertig behandelt wurden. Ein Algorithmus, der Zyklerkennung und Levelvergabe vermischt, muss rückwirkend an Stellen ändern, die er eigentlich schon verlassen hat.

3.2 Reihenfolgeabhängige Ergebnisse

Klassen kommen aus Jars. Die Reihenfolge, in der ein Tool Klassen sieht, folgt nicht der Architektur, sondern der Packreihenfolge des Jars, der Modulreihenfolge, dem Build-Tool oder Details des Dateisystems. Wenn ein Algorithmus Zyklen während dieser Traversierung nebenbei behandelt, hängt das Ergebnis von dieser zufälligen Reihenfolge ab.

Das ist gefährlich, weil der Fehler nicht sofort sichtbar sein muss. Die Grafik kann weiterhin ungefähr richtig aussehen. Trotzdem kann dieselbe Codebasis je nach Build-Konfiguration andere Level bekommen.

3.3 Vermischte Klassen- und Paketlevel

Ein naheliegender Fehler ist, das Level eines Pakets aus dem höchsten Level seiner enthaltenen Klassen abzuleiten. Das klingt plausibel, vermischt aber zwei verschiedene Fragen: Wo liegt eine Klasse im Klassengraphen, und wo liegt ein Paket in der Architekturhypothese?

Ein Paket ist nicht deshalb architektonisch hoch, weil es zufällig eine hoch eingeordnete Klasse enthält. Die zentrale Regel lautet:

Paketlevel entstehen aus Paketabhängigkeiten, nicht aus der maximalen Höhe einzelner Klassen.

Das klingt trivial, ist es aber nicht. Gerade weil Pakete Container für Klassen sind, liegt es nahe, ihre Position aus den enthaltenen Klassen abzuleiten. Genau dadurch vermischt man jedoch zwei Ebenen, die getrennt bleiben müssen. Erst wenn Paketlevel aus Paketabhängigkeiten entstehen, bleiben Pakete stabile architektonische Einheiten. Sonst genügt eine einzige Ausreißer-Klasse, um das Paket auf die falsche Ebene zu heben – und damit den Container unbrauchbar zu machen, der eigentlich Orientierung bieten soll.

3.4 Der große See

Die formal saubere Antwort auf Zyklen lautet: Alle Elemente einer SCC bekommen dasselbe Level. Für kleine Zyklen ist das akzeptabel. In realen Systemen können SCCs aber sehr groß werden. Dann landen Hunderte oder Tausende von Klassen auf derselben Ebene. Die Schichtung verschwindet; übrig bleibt ein flacher Block.

Damit wird der Zyklus zwar korrekt erkannt, aber nicht mehr brauchbar visualisiert. Ein Werkzeug muss große SCCs daher aufbrechen können. Entscheidend ist, dass die geschnittenen Kanten nicht verschwinden. Sie müssen sichtbar bleiben, weil genau sie erklären, warum die berechnete Ordnung nur als Architekturhypothese gilt.

4 Der Paketgraph als Architekturhypothese

Die von S202 gewählte Lösung besteht darin, Analyse, Ordnungsbildung und Darstellung konsequent zu trennen. Der Klassengraph bleibt das Rohmodell der tatsächlichen Abhängigkeiten. Er wird nicht verändert, nur weil für eine Schichtendarstellung ein DAG benötigt wird. Stattdessen erzeugt S202 aus diesem Graphen zusätzliche Sichten: SCCs für die Zyklererkennung, einen gewichteten Paketgraphen für die Architekturhypothese und eine hierarchische UI-Struktur für die Darstellung.

Mit Architekturhypothese ist dabei keine behauptete Soll-Architektur gemeint. Gemeint ist eine aus den beobachteten Abhängigkeiten abgeleitete Ist-Architektur: die Ordnung, die dem vorhandenen Code und seinen Abhängigkeiten am engsten entspricht, ohne zu verbergen, wo diese Ordnung durch Zyklen oder Rückkanten gebrochen wird.

Die Paketordnung bildet dabei das zentrale Bindeglied. Sie wird nicht aus den höchsten Klassenlevels abgeleitet, sondern aus den aggregierten Paketabhängigkeiten. Diese Trennung ist der entscheidende Schritt: Klassen werden innerhalb ihrer Pakete geordnet, Pakete dagegen nach ihren eigenen Paketbeziehungen. Dadurch kann ein Paket als Container stabil bleiben, auch wenn einzelne Klassen darin hoch oder niedrig liegen. Gleichzeitig liefert die Paketordnung ein Kriterium, um Back-Edges nicht beliebig auszuwählen, sondern als konkrete Verletzungen der erwarteten Ordnung zu kennzeichnen.

Damit verschwinden die oben beschriebenen Fehlermodi nicht durch einen einzelnen Algorithmustrick, sondern durch eine klare Zuständigkeit der Berechnungsschritte: Erst werden Abhängigkeiten analysiert, dann wird eine plausible Ordnung bestimmt, danach wird die sichtbare Struktur aufgebaut, und zuletzt wird geprüft, ob die Darstellung ihre eigene Aussage einhält.

5 Von Klassenabhängigkeiten zur Paketordnung

Der flache Klassengraph ist in S202 nicht das sichtbare Layoutmodell. Er ist ein Analyseartefakt: Er dient dazu, konkrete Abhängigkeiten zu erfassen, SCCs zu erkennen, Klassenabhängigkeiten zu Paketbeziehungen zu aggregieren und daraus die späteren Levels abzuleiten. Die sichtbare S202-Grafik entsteht danach aus der Paketstruktur und der lokalen Ordnung innerhalb der Paketcontainer.

Die Paketordnung entsteht aus einem gewichteten gerichteten Graphen über *alle* Pakete. Klassen sind in diesem Schritt keine eigenen Vergleichsknoten; sie liefern die Information für die Aggregation. Für jede Klassenabhängigkeit zählt S202 die Methodenaufrufe von der Quellklasse zur Zielklasse und ordnet diese Anzahl der entsprechenden Paketkante zu: Wenn eine Klasse im Teilbaum von Paket P eine Klasse im Teilbaum von Paket Q über n Methodenaufrufe aufruft, erhält die Kante $P \rightarrow Q$ das Gewicht n . Für strukturelle Abhängigkeiten ohne Aufrufzählung wird ein Gewicht von 1 verwendet.

Um den Beitrag verschachtelter Pakete zu erfassen, wird jedes Gewicht die Ahnenkette des Quellpakets hochpropagiert: Ein Aufruf von `com.a.X` nach `com.b.Y` trägt das Gewicht zu `com.a`→`com.b` und außerdem zu `com`→`com.b` bei. Die transitive Ordnung der Pakete entsteht erst bei der nachfolgenden Levelberechnung – dem Graphen werden keine zusätzlichen transitiven Kanten hinzugefügt.

Vor der Levelberechnung muss der Paketgraph azyklisch sein. Bilden Pakete eine SCC, bewertet S202 mit einem Rangmaß, ob ein Paket eher andere Pakete nutzt oder eher von ihnen genutzt wird:

$$rank(P) = \frac{out(P) - in(P)}{\max(1, out(P) + in(P))}$$

$out(P)$ ist die Summe der Kantengewichte von P zu anderen Mitgliedern derselben SCC, $in(P)$ die Summe der Kantengewichte aus diesen Mitgliedern nach P . Ein positiver Rang bedeutet: Das Paket benutzt mehr, als es selbst benutzt wird; es gehört tendenziell nach oben. Ein negativer Rang bedeutet: Das Paket wird eher benutzt und gehört tendenziell nach unten.

In einer Paket-SCC werden alle Kanten $P \rightarrow Q$ als Back-Edges behandelt, für die $rank(P) <$

$rank(Q)$ gilt – also alle Kanten, die gegen den Abhängigkeitsstrom laufen. Es wird kein Schwellwert verwendet: Jede messbare Rankdifferenz reicht als Begründung für einen Schnitt.

Sind alle Ranks innerhalb einer SCC gleich, liefert die Topologie keine Richtungsinformation. In diesem Fall werden alle internen Kanten entfernt und die Levels der betroffenen Knoten entstehen ausschließlich aus Abhängigkeiten außerhalb der ehemaligen SCC. Gibt es auch dort keine Abhängigkeiten, bleiben die Knoten auf demselben Level – als echte Peers ohne begründbare Ordnung.

Die Symmetrierkennung beruht auf dem Rankmaß, nicht auf einem Gewichtsvergleich. Eine SCC, in der alle Kanten dasselbe Gewicht tragen, ist nicht zwingend symmetrisch: Unterschiedliche Ein- und Ausgangsgrade erzeugen trotz gleicher Gewichte verschiedene Ranks und damit eine klare Schnittrichtung.

Nach dieser Zyklusbehandlung berechnet S202 die Levels auf dem resultierenden DAG. Level 0 bilden die Pakete, die keine weiteren Pakete benutzen. Jedes andere Paket liegt eine Ebene über dem höchsten Paket, das es direkt benutzt. Dadurch entsteht die transitive Schichtung: Aus $B \rightarrow A$ und $C \rightarrow B$ folgt A auf Level 0, B auf Level 1 und C auf Level 2, auch wenn keine direkte Kante $C \rightarrow A$ existiert.

Die hier erzeugten Levels sind das *architectureLevel* jedes Pakets – die Architekturhypothese, die im nächsten Schritt die Klassen-SCC-Behandlung steuert. In einem separaten Schritt, der nach den Klassenlevels ausgeführt wird, wendet S202 denselben Rank-Score-Algorithmus innerhalb jedes Parent-Containers unabhängig an. Dabei wird ein geschwistergewichteter Graph der direkten Kinder aufgebaut und ein *localLevel* für jedes Element abgeleitet. Das LocalLevel steuert die visuelle Position innerhalb des Parent-Containers und hat keinen Einfluss auf die Architekturhypothese oder die Verletzungserkennung.

6 Vom Klassenzyklus zur Paketordnung

Abbildung 3 zeigt ein zyklisches Beispiel aus dem Example-Jar als klassischen Klassengraphen. Die beiden Teilgraphen für Notification und Payment enthalten jeweils Rückkopplungen.

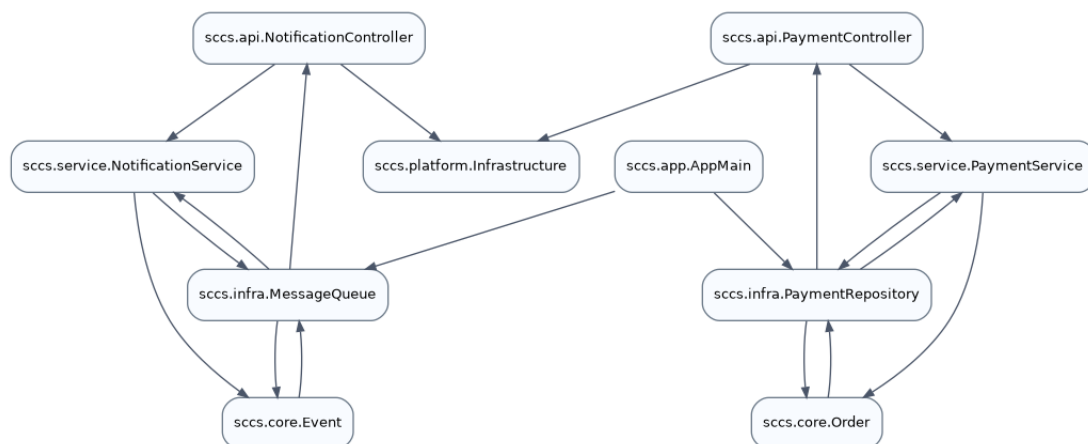


Abbildung 3: Zyklisches Beispiel aus `test-example-1.0.0.jar`. In jedem Zyklus muss mindestens eine Kante für die Levelberechnung geschnitten werden.

Die markierten zyklischen Bereiche sind zwei nicht-triviale SCCs: eine Notification-SCC mit `NotificationController`, `NotificationService`, `MessageQueue` und `Event`, sowie eine Payment-SCC mit `PaymentController`, `PaymentService`, `PaymentRepository` und `Order`. Das

etablierte Verfahren zur Ermittlung solcher Komponenten ist der Tarjan-Algorithmus.

Damit stellt der Klassengraph im Zyklusfall die technische Frage: Welche Kante muss man ignorieren, damit aus der SCC wieder ein berechenbarer DAG für die Levelbildung wird? Der Klassengraph allein liefert dafür aber kein fachliches Kriterium. Die Paketordnung liefert dieses Kriterium durch zwei Fälle.

Fall A betrifft SCCs, deren Mitglieder über verschiedene Paketlevels verteilt sind. Eine Kante $A \rightarrow B$ innerhalb einer solchen SCC wird geschnitten, wenn $pkgLevel(A) < pkgLevel(B)$ gilt – sie läuft von einem tiefer liegenden Paket in ein höher liegendes, gegen die architektonische Richtung.

Fall B betrifft SCCs, die vollständig auf einem einzigen Paketlevel liegen – kein Paketkontext liefert eine Richtung. Alle internen Kanten einer solchen SCC werden entfernt. Die Klassenlevels der ehemaligen Zyklusteilnehmer entstehen dann aus ihren Abhängigkeiten außerhalb der SCC; Knoten ohne solche externen Abhängigkeiten enden auf demselben Level.

Alle in beiden Fällen entfernten Kanten werden als *classBackEdge*-Befunde erfasst und bleiben als gestrichelte Verletzungskanten sichtbar.

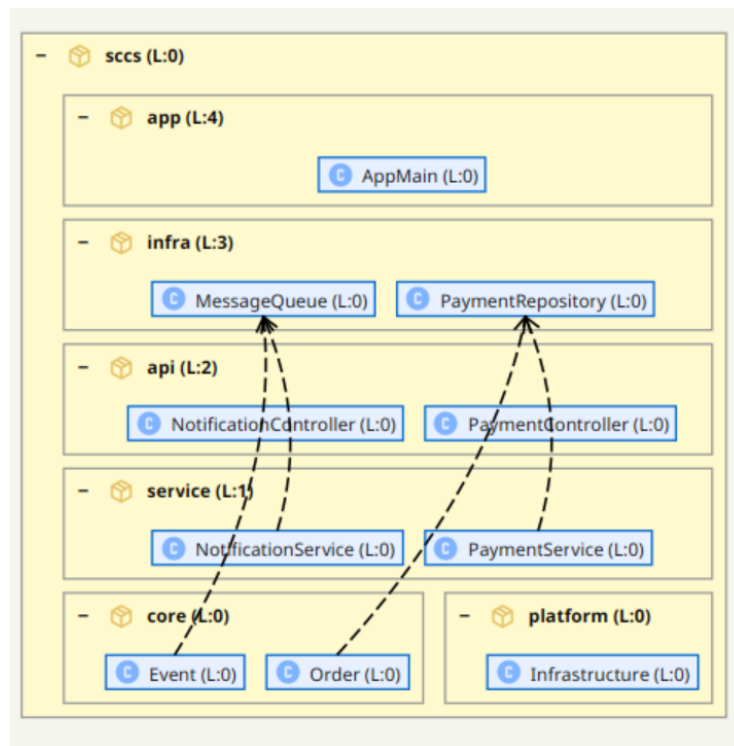


Abbildung 4: Zyklisches Beispiel in der S202-Darstellung. Die gestrichelten Kanten sind Verletzungen: Sie laufen gegen die aus den Paketabhängigkeiten abgeleitete Ordnung und bleiben als sichtbarer Befund der Architekturhypothese erhalten. Alle vier gestrichelten Kanten gehören zu den zwei SCCs und werden von S202 als Back-Edges für die Levelberechnung klassifiziert.

Die Begriffe Back-Edge und Verletzung beschreiben unterschiedliche Rollen:

Back-Edge Eine ausgewählte Kante innerhalb einer SCC, die für die Levelberechnung gesondert behandelt wird, damit der Berechnungsgraph azyklisch wird. Sie gehört zur Zyklusbehandlung.

Verletzung Eine konkrete Abhängigkeit, die nach der berechneten Ordnung nach oben läuft. Sie kann aus einer SCC stammen, muss es aber nicht. Verletzungen sind damit eine Ober-

menge der Back-Edges: Eine Back-Edge kann als Verletzung sichtbar werden, aber nicht jede Verletzung ist eine Back-Edge. Begrifflich bleibt eine Verletzung die sichtbare Ordnungsabweichung, während eine Back-Edge die Rolle der Kante in der Zyklusbehandlung beschreibt.

S202 versteckt den Zyklus daher nicht. Als Back-Edges klassifizierte Kanten bleiben als Verletzungen sichtbar, weil sie erklären, warum die Paketordnung nicht vollständig zu den konkreten Klassenabhängigkeiten passt. Dadurch wird die Abweichung nachvollziehbar statt beliebig.

7 Redundante Prüfung der Konsistenz

Die bisher beschriebenen Schritte zeigen bereits, warum die Umsetzung fehleranfällig ist. Es reicht nicht, einzelne Algorithmen isoliert mit Unit Tests abzusichern. Viele Fehler entstehen erst durch das Zusammenspiel der Daten: konkrete Klassenabhängigkeiten, Paketstruktur, SCCs, aggregierte Paketgewichte und lokale Darstellungspositionen beeinflussen sich gegenseitig.

S202 prüft deshalb nicht nur einzelne Rechenschritte. Nachdem die komplette Pipeline gelaufen ist, wird das fertige Ergebnis redundant gegen Konsistenzregeln geprüft. Diese Regeln berechnen das Layout nicht neu und korrigieren auch nichts. Sie lesen nur das Ergebnis und fragen: Ist die erzeugte Darstellung in sich stimmig?

Die Prüfung betrachtet drei Ebenen:

1. **Klassengraph:** Für normale Klassenabhängigkeiten muss gelten: Wenn $A \rightarrow B$, dann steht A auf einem höheren Level als B . Ausnahmen müssen explizit klassifiziert sein, zum Beispiel als Back-Edge oder als Kante innerhalb einer SCC.
2. **Paketgraph:** Die berechnete Paketordnung muss konsistent mit den gewichteten Paketabhängigkeiten sein. Für jede Paketabhängigkeit $P \rightarrow Q$, die nicht als Paket-Back-Edge klassifiziert ist, muss P strikt oberhalb von Q in der berechneten Ordnung liegen. Kanten, die beim Aufbrechen einer Paket-SCC geschnitten werden, müssen als Paket-Back-Edges klassifiziert sein und sind von dieser Prüfung ausgenommen.
3. **Sichtbare Darstellung:** Die sichtbare Anordnung muss zum berechneten Modell passen. Paketcontainer, lokale Klassenpositionen und verschachtelte Levels dürfen die berechnete Architekturhypothese nicht stillschweigend umkehren.

Diese Konsistenzregeln sind keine zusätzliche Optimierung und kein zweiter Layoutalgorithmus. Sie sind ein unabhängiger Plausibilitätscheck nach Abschluss der Berechnung. Genau dadurch finden sie Fehler, die in normalen Unit Tests leicht fehlen: ein Paketlevel wurde korrekt berechnet, aber lokal falsch dargestellt; eine Back-Edge wurde geschnitten, aber nicht als solche markiert; eine normale Kante läuft nach oben, ohne dass sie Teil einer SCC ist.

Der entscheidende Punkt ist Redundanz. Die Pipeline erzeugt die Darstellung. Die Regeln prüfen anschließend, ob diese Darstellung ihre eigene Aussage einhält. Damit wird aus einer reinen Grafik ein überprüfbares Modell.

8 Von der Visualisierung zur Aktion

Eine berechnete Architekturhypothese ist besonders dann nützlich, wenn sie nicht nur einen Zustand beschreibt, sondern Veränderung planbar macht. S202 enthält dafür eine What-If-Analyse:

Klassen und Pakete können per Drag and Drop in der Darstellung verschoben werden. Das Modell wird dadurch nicht sofort refaktoriert; stattdessen berechnet das Werkzeug, welche Abhängigkeiten in der neuen Anordnung nach oben laufen würden.

Die Verletzungen aktualisieren sich nach jeder Verschiebung. Dadurch wird sichtbar, ob eine geplante Umordnung die Architektur konsistenter macht oder neue Probleme erzeugt. Das ist vor allem in Situationen wichtig, in denen das Werkzeug keine eindeutige Ordnung mehr findet: große SCCs, symmetrische Paketabhängigkeiten oder historisch gewachsene Kopplungen lassen sich nicht immer automatisch sinnvoll auflösen.

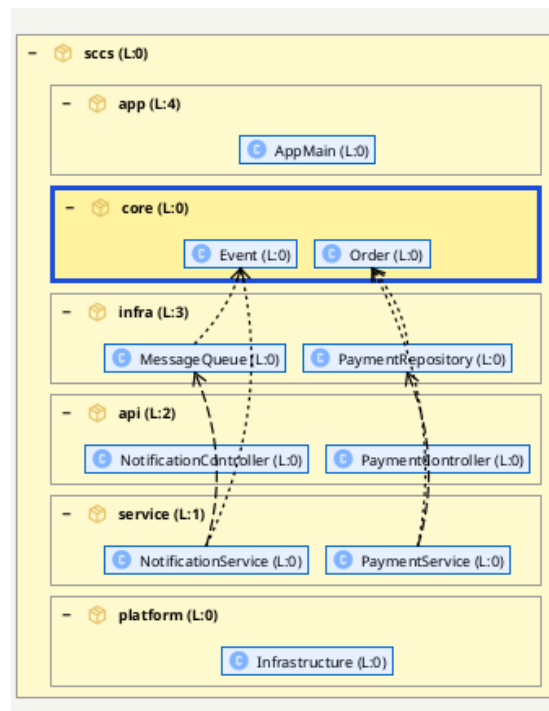


Abbildung 5: What-If-Verschiebung: Das Paket `core` wird aus Level 0 zwischen Level 3 und Level 4 verschoben. Dadurch entstehen sechs Verletzungen, weil bestehende Abhängigkeiten nun gegen die gewählte Anordnung laufen.

Abbildung 5 zeigt genau diese Arbeitsweise: Die manuelle Anordnung entspricht nicht mehr der aus den Abhängigkeiten berechneten Ordnung. Sie kann trotzdem eine gewünschte Zielarchitektur ausdrücken, etwa wenn `core` fachlich oberhalb der Infrastruktur liegen soll. S202 behandelt diese Entscheidung deshalb nicht als Fehler der Darstellung, sondern macht ihre Kosten sichtbar: In diesem Fall müssten sechs konkrete Beziehungen refaktoriert oder entkoppelt werden, damit die Abhängigkeiten wieder zur gewählten Schichtung passen.

Ergänzend dazu gibt es eine manuelle CUT-Logik. Back-Edges können gezielt als geplante Schnitte markiert werden, und das Werkzeug kann anschließend prüfen, ob diese Schnitte die betroffene SCC tatsächlich auflösen. Der Benutzer sieht nicht nur, welche Kante als Bruch der Ordnung plausibel wäre, sondern kann die Wirkung dieses Schnitts auf den Zyklus überprüfen.

Diese CUT-Logik arbeitet feiner als die sichtbare Klassen- oder Paketkante. Geschnitten wird auf Methodenaufabeebene: Nicht zwingend die gesamte Beziehung zwischen zwei Klassen verschwindet, sondern der konkrete Methodenaufwurf, der die zyklische Abhängigkeit verursacht. Dadurch passt das Feature besser zur praktischen Refactoring-Arbeit, weil ein Zyklus oft durch wenige konkrete Aufrufe entsteht, nicht durch die komplette fachliche Beziehung zweier Klassen.

9 Umsetzung auf konzeptioneller Ebene

Die Umsetzung trennt die Berechnung bewusst in mehrere Phasen. Entscheidend ist, dass keine Phase eine sichtbare Position festlegt, bevor die Informationen vorliegen, die diese Position begründen.

1. **Klassengraph aufbauen:** Aus dem Bytecode wird ein gerichteter Klassengraph erzeugt. Dieser Graph bleibt das fachliche Rohmodell der Analyse.
2. **SCCs erkennen:** Auf dem Klassengraphen werden stark zusammenhängende Komponenten mittels Tarjan ermittelt. Das Ergebnis ist noch kein Layout, sondern die Information, welche Klassen azyklisch einordenbar sind und welche zu Zyklen gehören.
3. **Paketgraph ableiten:** Aus den Klassenabhängigkeiten werden Paketabhängigkeiten zu einem gewichteten globalen Paketgraphen aggregiert. Jeder Methodenaufwurf von einer Klasse in Paket P zu einer Klasse in Paket Q trägt zum Gewicht der Kante $P \rightarrow Q$ bei, propagiert die Ahnenkette von P aufwärts. Das Ergebnis ist ein einziger flacher Graph über alle Pakete.
4. **Paketordnung berechnen:** Aus den gewichteten Ein- und Ausgängen wird das Rangmaß $rank(P)$ berechnet. Alle Kanten, für die $rank(P) < rank(Q)$ gilt, werden als Back-Edges klassifiziert und in einem Durchlauf geschnitten. Sind alle Ranks innerhalb einer SCC gleich, werden alle internen Kanten entfernt; Levels entstehen dann aus Abhängigkeiten außerhalb der ehemaligen SCC.
5. **Klassen-Back-Edges bestimmen:** Klassen-SCCs werden mithilfe der Pakethypothese aufgebrochen. *Fall A* – eine Kante $A \rightarrow B$ innerhalb einer SCC wird geschnitten, wenn $pkgLevel(A) < pkgLevel(B)$ gilt: Sie läuft gegen die architektonische Richtung. *Fall B* – wenn alle SCC-Mitglieder dasselbe Paketlevel haben, liefert kein Paketkontext eine Richtung; alle internen Kanten werden entfernt. In beiden Fällen bedeutet *Schneiden*, dass die Kante vom DAG der Levelberechnung ausgenommen, aber als *classBackEdge*-Befund erhalten bleibt und als gestrichelte Verletzungskante sichtbar ist.
6. **Lokale Darstellungspositionen zuweisen:** Nachdem die Klassenlevels vergeben sind, wird das *localLevel* jedes Elements berechnet. Innerhalb jedes Parent-Containers wird ein geschwistergewichteter Graph der direkten Kinder aufgebaut; derselbe Rank-Score-Algorithmus bricht etwaige Zyklen auf; Longest-Path ergibt dann ein *localLevel* pro Kind. Das *localLevel* bestimmt die visuelle Position innerhalb der Parent-Box; das *architectureLevel* ist das globale semantische Level, das die Verletzungserkennung steuert.
7. **Konsistenz prüfen:** Zum Schluss wird die fertige Darstellung redundant gegen die Konsistenzregeln geprüft. Normale Abhängigkeiten, SCCs, Back-Edges und Verletzungen werden unterschieden und klassifiziert.

Kurz gefasst:

```
Bytecode -> Klassengraph -> SCCs -> Paketgraph -> Paket-  
    architectureLevel  
    -> Klassen-architectureLevel -> localLevel ->  
    Konsistenzregeln
```

10 Fazit

S202 berechnet aus den bestehenden Abhängigkeiten die plausibelste Hypothese der Ist-Architektur. Das Ergebnis ist eine überprüfbare hierarchische Schichtendarstellung: Klassenabhängigkeiten liefern den Graphen, Paketabhängigkeiten liefern eine belastbare Ordnung, Back-Edges machen notwendige Brüche sichtbar, und Konsistenzregeln prüfen, ob die fertige Darstellung ihre eigene Aussage einhält.

Der wichtigste Beitrag liegt deshalb nicht in einer optisch verbesserten Grafik, sondern in der Verbindung von Darstellung, Erklärung und Veränderbarkeit. Die Schichtung zeigt eine mögliche Ordnung. Die markierten Befunde erklären, wo diese Ordnung durch konkrete Abhängigkeiten gebrochen wird. What-If-Verschiebungen und manuelle CUTs machen sichtbar, welche Refactorings diese Brüche reduzieren oder Zyklen tatsächlich auflösen können.

Damit ist die Visualisierung mehr als ein Layout. Sie ist ein erklärbares Arbeitsmodell: Jede Position und jede hervorgehobene Kante muss auf eine nachvollziehbare Entscheidung in der Berechnung zurückführbar sein, und jede geplante Änderung kann gegen dieselbe Ordnung geprüft werden.